Taylor-Couette Mixing Research Log 2

Isaiah Bascom

March 25th 2026

1 Introduction

Since last meeting I've worked on:

- Some RL Exercise in the OpenAI Gymnasium.
  - Playing blackjack. [1]
  - Balancing a pole on a cart [2]

2 Exercising at the Gym

I had some fun using the Gymnasium which is the latest version of OpenAI's Gym. Initially I ran through the Basic Usage tutorial learning from its implementation of Q-learning in Blackjack. Then I moved to the Lunar Lander environment to try Deep Q-learning. You can find my code for each here https://github.com/ibascom7/tc-demos

2.1 Blackjack

Key notes for this project:

- Uses classic Q-learning algorithm as laid out in Brunton's

  $$Q^{\text{new}}(\mathbf{s}_k, \mathbf{a}_k) = Q^{\text{old}}(\mathbf{s}_k, \mathbf{a}_k) + \alpha \left(r_k + \gamma \max_{\mathbf{a}} Q(\mathbf{s}_{k+1}, \mathbf{a}) - Q^{\text{old}}(\mathbf{s}_k, \mathbf{a}_k) \right).$$

  Interestingly, the implementation can be simplified to

  ```
  future_q_value = (not terminated) * np.max(self.q_values[next_obs])
  target = reward + self.discount_factor * future_q_value
  temporal_difference = target - self.q_values[obs][action]
  self.q_values[obs][action] = (
      self.q_values[obs][action] + self.lr * temporal_difference
      )
  ```

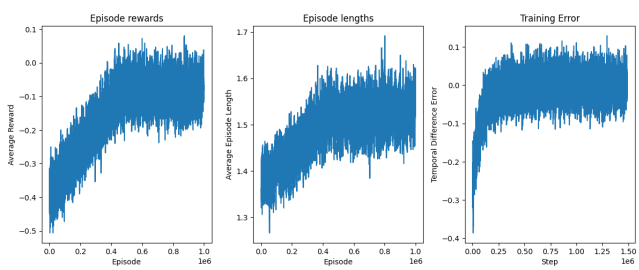

Figure 1: Blackjack Q-learning training results.

- Implements epsilon-greedy strategy, with a random chance for exploration

vs exploitation.

$$\varepsilon = \text{randomly choose action}$$

$$1 - \varepsilon = \text{choose action based off of our Q-table}$$

$$\varepsilon_{start} = 1.0$$

$$\varepsilon_{final} = 0.1$$

$$\varepsilon_{decay} = \frac{\varepsilon_{start}}{n_{episodes}/2}$$

2.2 Cart Pole

Key notes for this project:

- Uses Deep Q-learning algorithm as seen in PyTorch's Documentation [4]. The secret here is that it follows the same Q-learning process, but since our state space in this example is continuous, it must approximate the Q-table with a neural network. Here, $\boldsymbol{\theta}$ represents the network's weights.

$$Q(\mathbf{s}, \mathbf{a}) \approx Q(\mathbf{s}, \mathbf{a}, \boldsymbol{\theta})$$

- Similarly to the Blackjack implementation, we use a epsilon-greedy strategy. However, in this case instead of epsilon decaying linearly it decays exponentially. We make this choice because the neural network can generalize to many new states, so it doesn't need to see as many new states. It can learn more generally that if the pole is falling to the left it should move to the left and vice versa.

- Random Sampling was used during training. The main loop travels using an epsilon-greedy strategy and then runs optimize model which takes a batch of the last 128 actions. The random batch is what is used to train the model which improves generalization and stability by removing temporal dependency.

- A very surprising discovery was how unstable Deep Q-learning can be. While using the same hyperparameters as the tutorial, different runs tended to vary quite a bit from each other. A part of this is the initial randomness can mean some runs have better early training data than others.

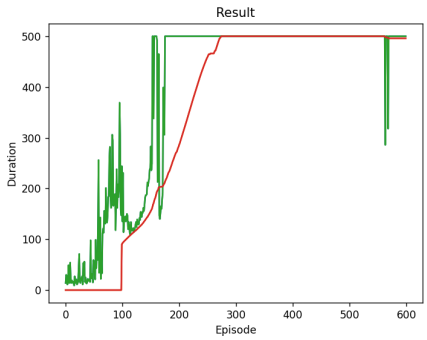

Figure 2: Best run. Very similar to the example from the tutorial

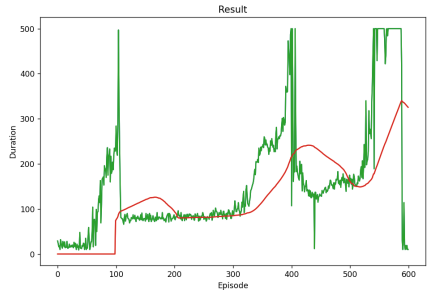

Figure 3: Decent run.

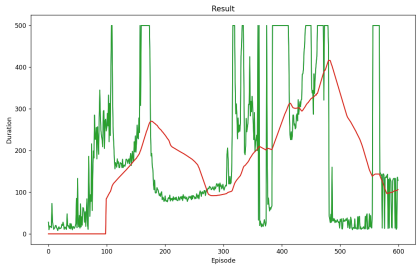

Figure 4: Bad run.

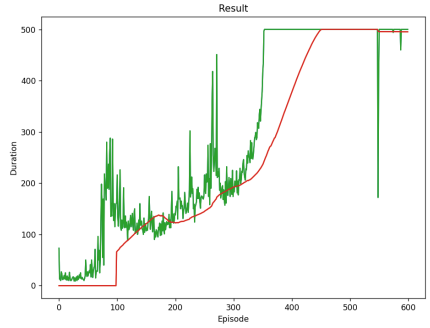

Figure 5: Latest run.

3 Next Steps

- Recreate TD3 half-cheetah experiment and mathematically describe the implementation.
- Play with other spinning up algorithms in Gymnasium environments
- Make a dummy OpenFOAM case for a simpler test of TD3.

References

[1] Farama Foundation, "Basic Usage," *Gymnasium Documentation*. https://gymnasium.farama.org/introduction/basic_usage/

[2] CartPole Environment, *Gymnasium Documentation*, v0.28.1. https://gymnasium.farama.org/environments/classic_control/cart_pole/

[3] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing Function Approximation Error in Actor-Critic Methods," *arXiv preprint arXiv:1802.09477*, 2018. https://arxiv.org/abs/1802.09477

[4] Adam Paszke, "Reinforcement Learning (DQN) Tutorial," *PyTorch Documentation*. https://docs.pytorch.org/tutorials/intermediate/reinforcement_q_learning.html

[5] J. Achiam, "Twin Delayed DDPG (TD3)," *Spinning Up in Deep RL*, OpenAI. https://spinningup.openai.com/en/latest/algorithms/td3.html

[6] S. Fujimoto, "TD3: Official Implementation," *GitHub Repository*. https://github.com/sfujim/TD3